LAPORAN PRAKTIKUM SESI 10

KOMUNIKASI ANTARSERVICE VIA MESSAGE QUEUE

Penerapan RabbitMQ pada Marketplace UMKM

Mata Kuliah Web Service



Dosen Pengampu

Philipus Suryo Subandoro

Disusun oleh

Bambang Herlambang 5803024019

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

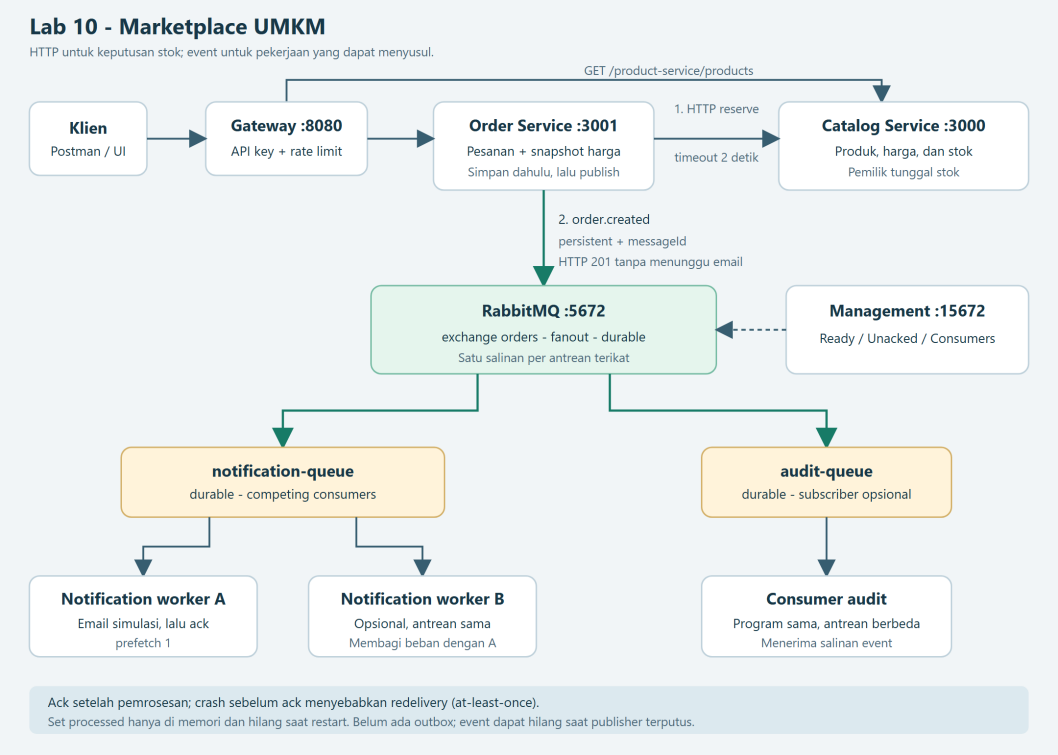
UNIVERSITAS KATOLIK WIDYA MANDALA

SURABAYA

2026

1. Implementasi dan arsitektur

Lab 10 menambahkan RabbitMQ dan Notification Service pada Marketplace UMKM. Alur sinkron dan asinkron disajikan dalam diagram berikut.



Gambar 1. Arsitektur Lab 10: reservasi stok sinkron dan notifikasi asinkron.

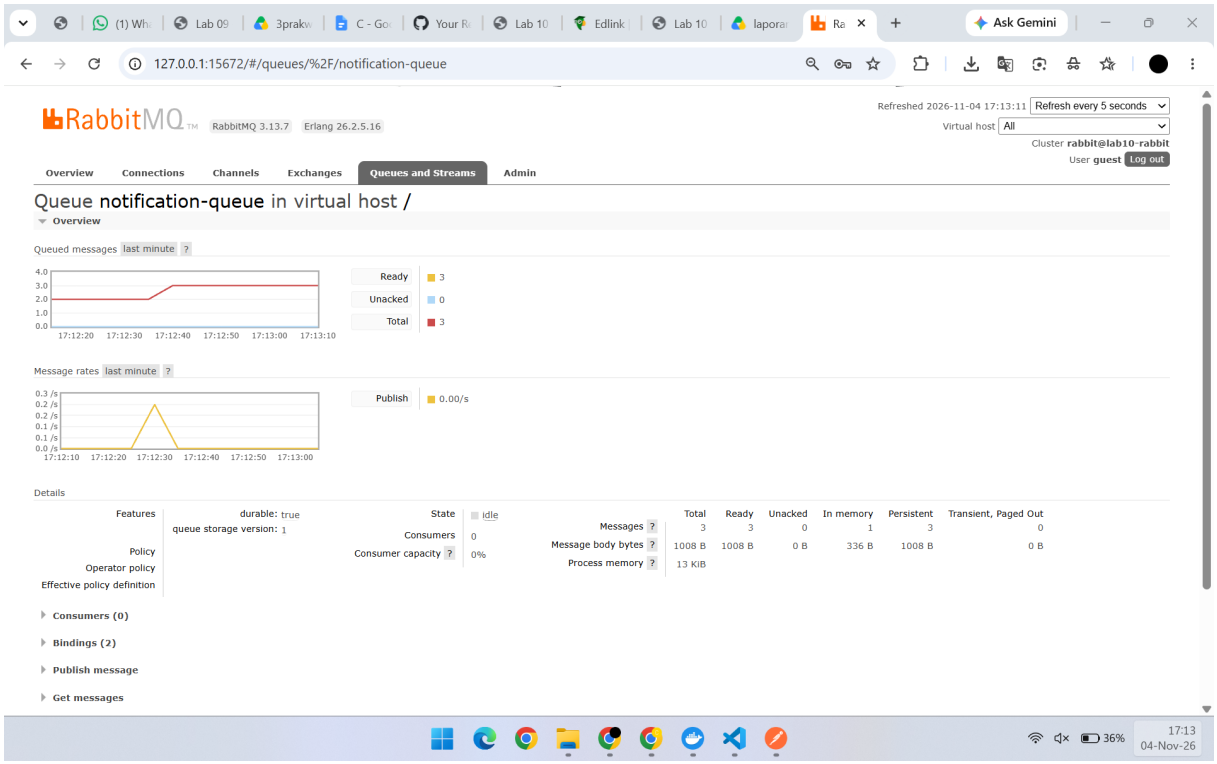
Pada Gambar 1, Order Service mereservasi stok melalui HTTP, lalu menerbitkan order.created ke RabbitMQ. Notification Service memproses email simulasi dan memberikan ack.

Komponen	Tanggung jawab
Catalog Service :3000	Pemilik produk, harga, dan stok; reserve melalui HTTP.
Order Service :3001	Menyimpan pesanan dan snapshot harga; producer order.created.
RabbitMQ :5672	Exchange orders fanout; antrean durable dan pesan persistent.
Notification Service	Consumer notification-queue; manual ack, prefetch 1, deduplikasi Set di memori.

Menjalankan: npm run broker:up; npm run start:notification; npm start. Pengujian melalui Postman/gateway :8080. Consumer dibuat sebelum order pertama agar antrean sudah terikat.

2. Consumer mati dan pesan menunggu

Consumer dihentikan, kemudian tiga order dikirim melalui gateway menggunakan Postman. Ketiganya mendapat HTTP 201 Created; responsnya ditampilkan pada Gambar 7-9 dalam lampiran.



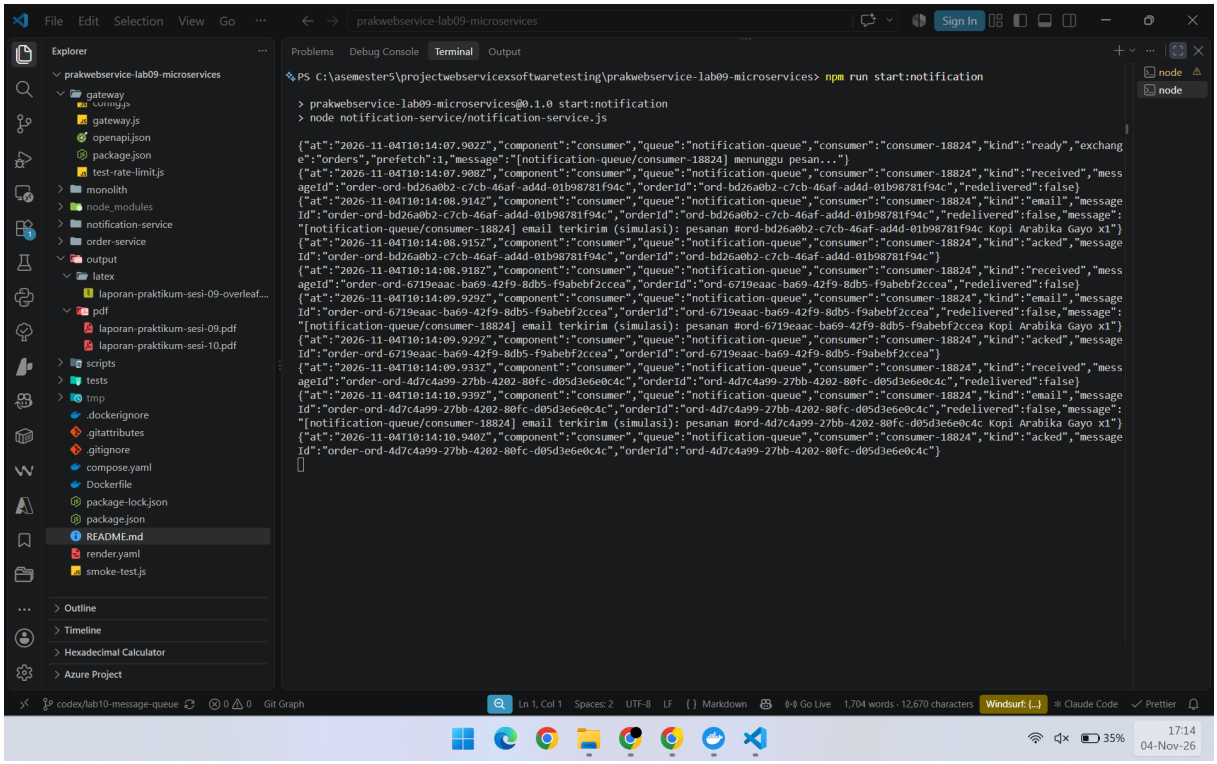
Gambar 2. Dashboard notification-queue, 4 November 2026 pukul 17:13:11 UTC+7.

Pada Gambar 2, Ready=3 menunjukkan tiga pesan menunggu, Unacked=0 berarti tidak ada pesan sedang diproses, dan Consumers=0 memastikan consumer mati. Broker tetap aktif pada vhost /.

Pengamatan	**Hasil**
Ready / Unacked / Consumers	3 / 0 / 0
Tiga request Postman	Masing-masing 201 Created

3. Consumer dinyalakan kembali

Setelah npm run start:notification dijalankan kembali, consumer-18824 memproses ketiga pesan tertahan. Setiap email simulasi diikuti ack; orderId cocok dengan ketiga respons Postman.



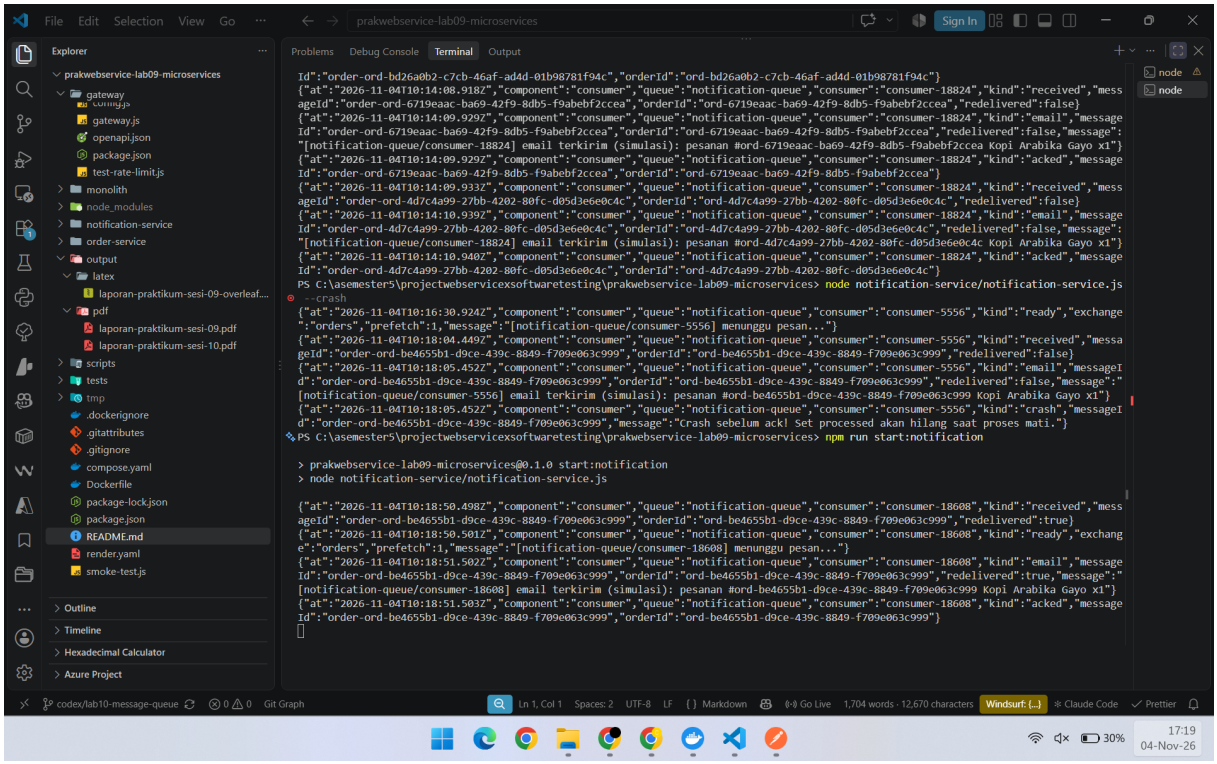
Gambar 3. Log consumer setelah dinyalakan kembali.

Pada Gambar 3, tiga ID pesanan pada Gambar 7-9 dalam lampiran diproses satu per satu. Setiap email simulasi diikuti ack, sehingga pesan yang menunggu diselesaikan setelah consumer pulih.

OrderId sesuai Postman	**Email / ack UTC+7**
ord-bd26a0b2-c7cb-46af-ad4d-01b98781f94c	17:14:08.914 / 17:14:08.915
ord-6719eaac-ba69-42f9-8db5-f9abebf2ccea	17:14:09.929 / 17:14:09.929
ord-4d7c4a99-27bb-4202-80fc-d05d3e6e0c4c	17:14:10.939 / 17:14:10.940

4. Crash sebelum ack dan redelivery

Consumer dijalankan dengan --crash, lalu satu order dibuat. Respons Postman ada pada Gambar 10 dalam lampiran. Setelah crash, consumer biasa dinyalakan tanpa membuat order baru.



Gambar 4. Log crash dan pemulihan consumer untuk pesan yang sama.

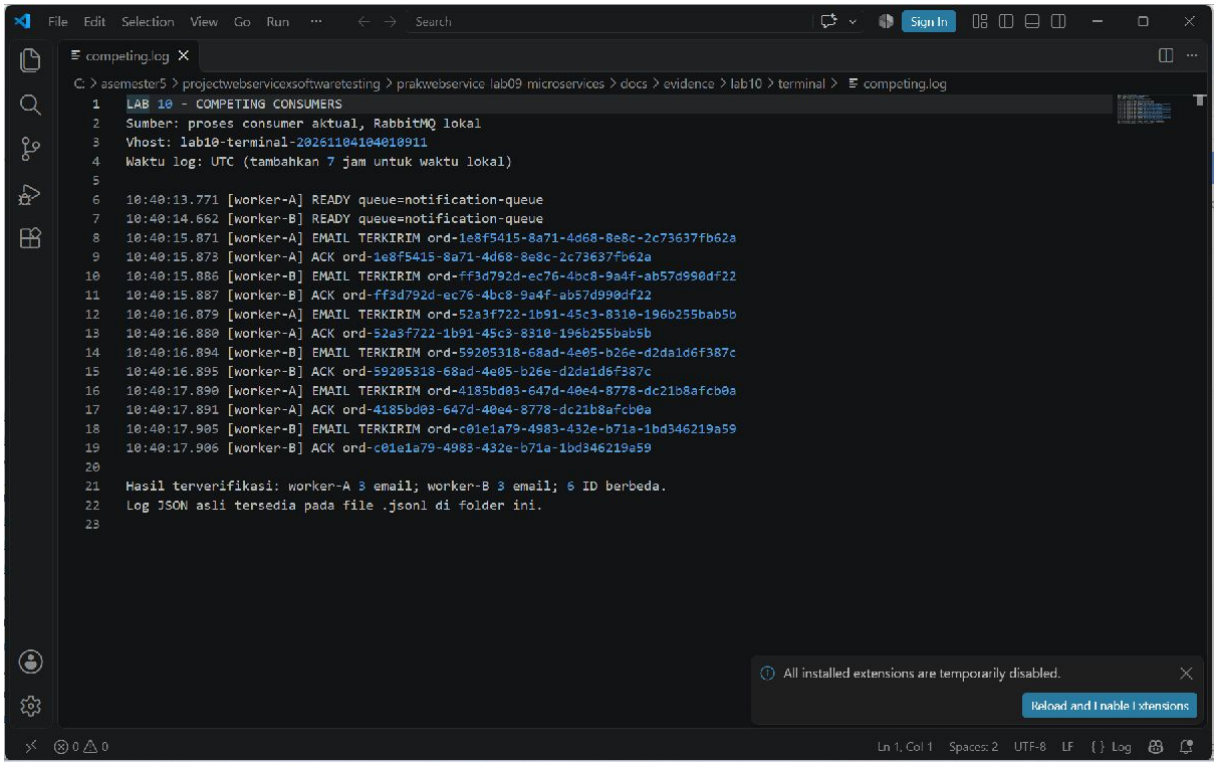
Pada Gambar 4, email pertama diikuti crash sebelum ack. Setelah restart, pesan yang sama mencetak email kedua dengan redelivered:true, lalu di-ack. Set processed hilang saat restart; pemrosesan bersifat at-least-once.

MessageId yang sama: order-ord-be4655b1-d9ce-439c-8849-f709e063c999

Tahap	Waktu UTC+7	Hasil
Email pertama	17:18:05.452	consumer-5556; redelivered:false; crash sebelum ack
Email kedua	17:18:51.502	consumer-18608; redelivered:true
Ack	17:18:51.503	consumer-18608; kind: acked

5. Competing consumers

Pengujian tambahan pukul 17:40 UTC+7 menjalankan dua consumer pada satu notification-queue. Enam order diterima melalui gateway dengan HTTP 201; event dibagi tanpa ID yang tumpang tindih.



Gambar 5. Screenshot VS Code atas ringkasan log consumer aktual; sumbernya file JSONL.

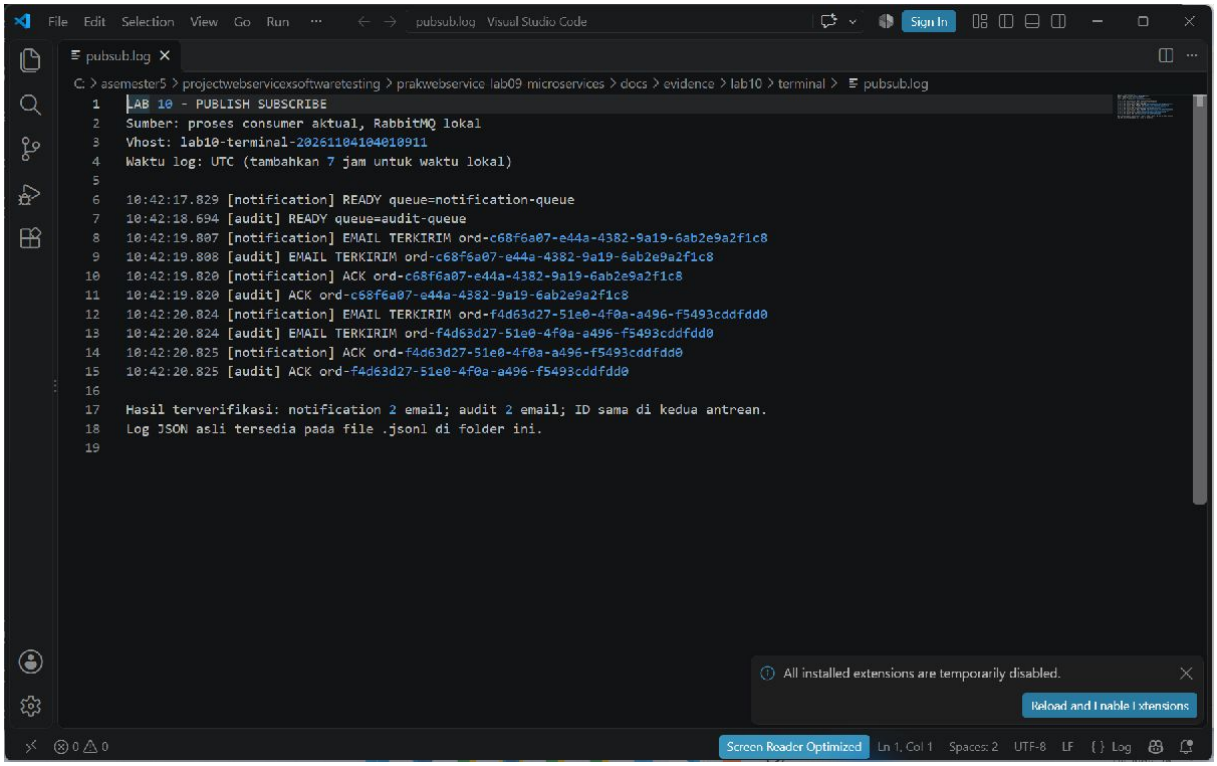
Pada Gambar 5, worker-A dan worker-B masing-masing memproses tiga event. ID yang berbeda pada kedua worker menunjukkan setiap event dikerjakan oleh satu consumer.

Consumer	Email simulasi	Ack
worker-A	3	3
worker-B	3	3

Prefetch 1 membatasi satu pesan unacked per worker. Pembagian 3+3 adalah hasil run ini, bukan jaminan pembagian selalu seimbang. Pengujian memakai vhost terpisah dari uji manual.

6. Publish subscribe

Pada pukul 17:42 UTC+7, consumer notification-queue dan audit-queue terikat ke exchange orders. Dua order baru dikirim; masing-masing antrean menerima kedua event dengan orderId yang sama.



Gambar 6. Screenshot VS Code atas log aktual notification dan audit.

Pada Gambar 6, dua orderId yang sama muncul pada consumer notification dan audit. Exchange fanout menyalin setiap event ke kedua antrean, kemudian masing-masing consumer memberi ack.

OrderId	Penerima
ord-c68f6a07-e44a-4382-9a19-6ab2e9a2f1c8	notification dan audit
ord-f4d63d27-51e0-4f0a-a496-f5493cddfdd0	notification dan audit

Competing consumers membagi pekerjaan pada antrean yang sama. Pub/sub memberi salinan event ke antrean berbeda. Consumer audit masih memakai program simulasi yang sama, belum database audit produksi.

7. Penerapan proyek dan refleksi

Aspek	Marketplace UMKM
Interaksi asinkron	Notifikasi pesanan baru setelah checkout.
Event dan isi pesan	order.created: ID pesanan, snapshot items/harga, quantity, total, status, waktu; messageId stabil.
Producer / consumer	Order Service / Notification Service; subscriber audit opsional.
Alasan	Email dapat menyusul; reservasi stok tetap sinkron karena menentukan penerimaan order.

Ringkasan pengujian

Suite Lab 09 dan Lab 10: **75/75 test lulus**, 0 gagal; delapan skenario integrasi RabbitMQ asli lulus. Duplikat dalam proses yang sama dilewati; JSON rusak di-nack tanpa requeue. Saat koneksi publisher ditutup, order tetap 201 tetapi event hilang; reconnect hanya memulihkan pengiriman order baru.

Refleksi (5 kalimat)

Jika RabbitMQ mati saat order dibuat, order tetap tersimpan dan klien menerima HTTP 201, tetapi event dapat hilang sehingga notifikasi tidak konsisten. Penyimpanan order bersama transactional outbox dan relay dengan retry diperlukan untuk mengurangi celah tersebut. Consumer idempotent perlu messageId unik di penyimpanan tahan restart serta transaksi yang mencatat ID dan side effect database sebelum ack. Email eksternal memerlukan outbox pengiriman dan dukungan idempotency key penyedia karena tabel processed saja tidak menjamin email tepat sekali. Panggilan sinkron tetap tepat untuk stok dan harga saat checkout karena hasilnya dibutuhkan segera, sedangkan notifikasi dapat diproses kemudian.

Batasan dan sumber

Stok, pesanan, dan Set deduplikasi masih in-memory; belum ada transactional outbox atau SMTP. Lab 10 diuji lokal melalui Docker. Bukti manual, run tambahan, dan arsip otomatis memiliki waktu/ID/vhost tersendiri. Berkas lengkap tersedia pada docs/evidence/lab10/.

Repository pengumpulan Lab 10.

RabbitMQ Work Queues

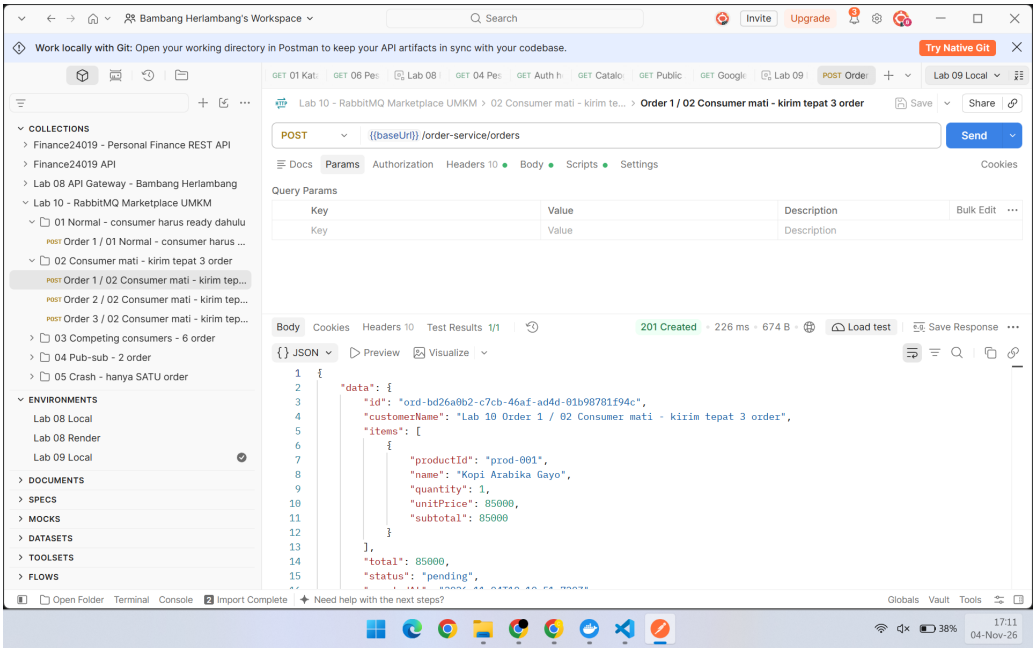
RabbitMQ Publish Subscribe

amqplib Channel API

Panduan Praktikum Web Service Lab 10, Philipus Suryo Subandoro.

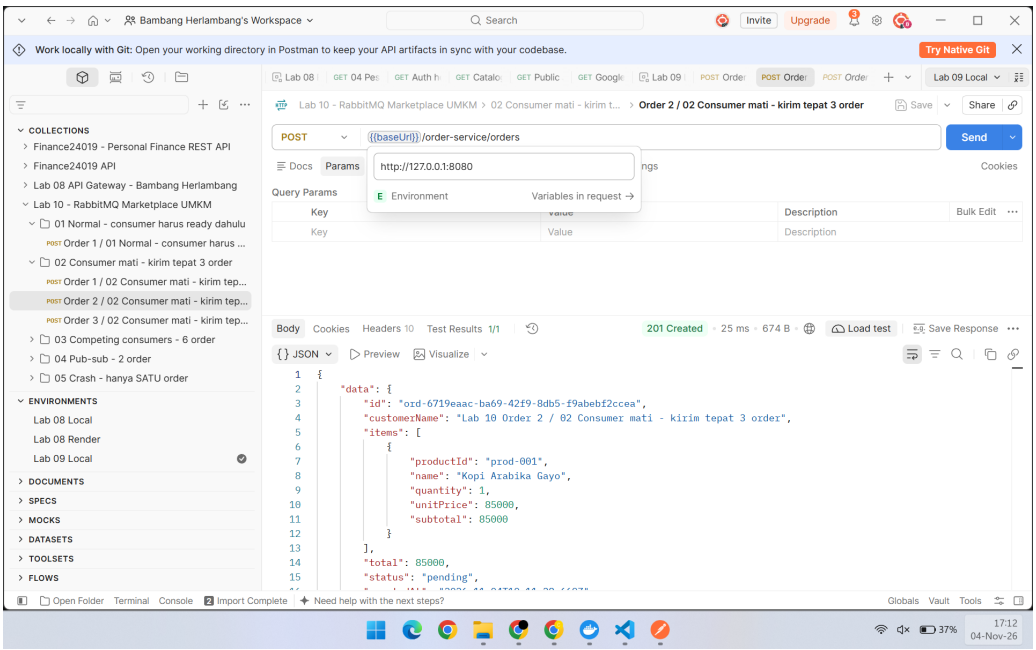
Lampiran. Bukti request Postman

Screenshot berikut memperlihatkan respons request manual. ID pesanan dicocokkan dengan log pemulihan pada Gambar 3 dan log crash pada Gambar 4.



Gambar 7. Respons Postman untuk pesanan pertama saat consumer mati.

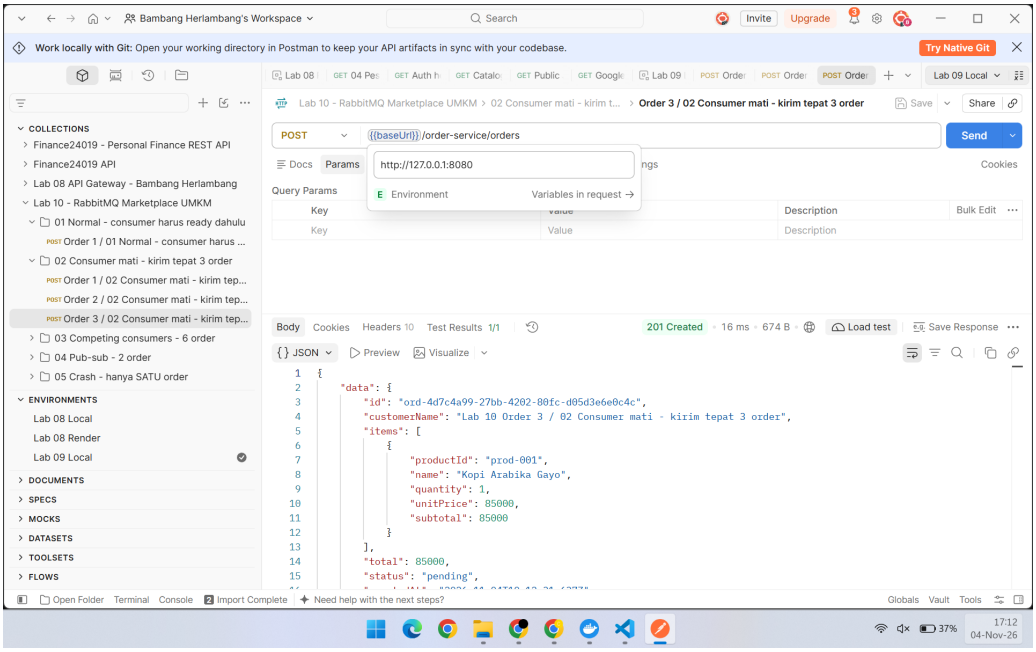
Pada Gambar 7, order pertama memperoleh 201 Created meskipun consumer berhenti. ID yang sama muncul pada log Gambar 3 setelah consumer dinyalakan kembali.



Gambar 8. Respons Postman untuk pesanan kedua saat consumer mati.

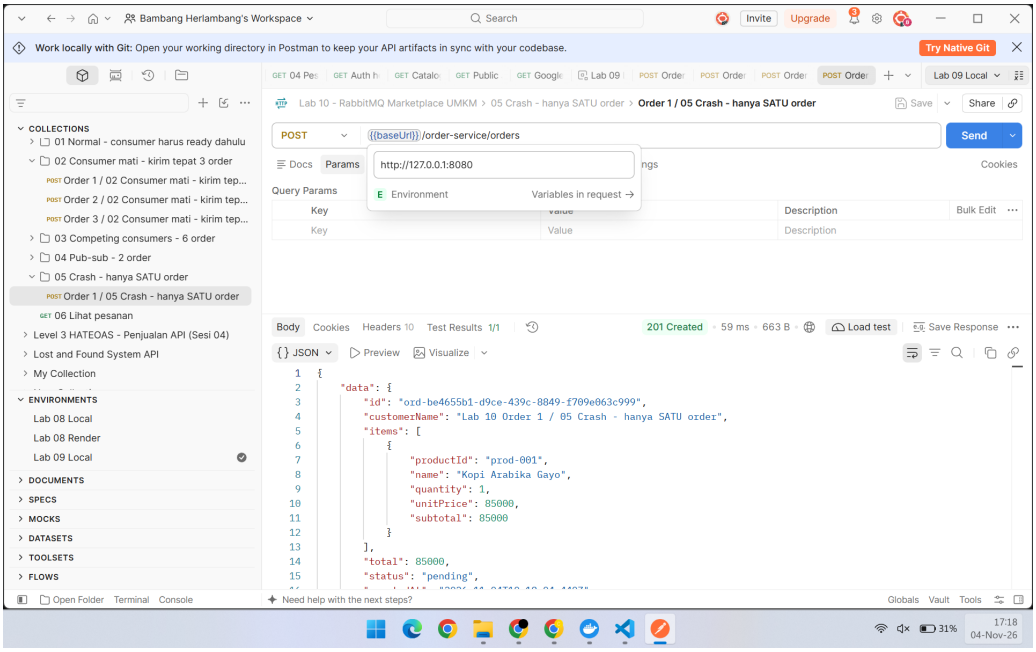
Pada Gambar 8, order kedua juga mendapat 201 Created dengan ID berbeda. Penerimaan order tetap berjalan, sedangkan notifikasi menunggu consumer aktif.

Lampiran. Bukti request Postman (lanjutan)



Gambar 9. Respons Postman untuk pesanan ketiga saat consumer mati.

Pada Gambar 9, order ketiga mendapat 201 Created. Tiga request pada Gambar 7-9 menghasilkan pesan yang menunggu, sesuai Ready=3 pada dashboard Gambar 2.



Gambar 10. Respons Postman untuk satu pesanan uji crash.

Pada Gambar 10, hanya satu order dibuat dan mendapat 201 Created. Dua email pada log Gambar 4 memakai ID pesanan yang sama, sehingga berasal dari redelivery pesan tersebut.